APPLICATION DOCUMENTATION

#Sustenido / LiveNLoud App Documentation

Internal product and technical documentation for the application in `/Users/eloygomes/Documents/GitHub/new-LiveNLoud`.

Generated on April 23, 2026. Frontend package version: **0.62.0.0**.

1. Product Overview

#Sustenido is a music preparation and performance workspace for musicians. The app helps users maintain a personal song library, organize songs into setlists, track learning progress by instrument, export playlists to music platforms, and use live performance tools such as presentation mode, metronome, tuner, and calendar scheduling.

The codebase still contains the historical LiveNLoud name in folders and backend identifiers, while the active product identity in the frontend is #Sustenido.

PRIMARY AUDIENCE

Individual musicians and collaborators who need to prepare, organize, rehearse, and perform song libraries across several instruments.

CORE WORKFLOW

Add songs, enrich them with links and instrument data, organize them into setlists, filter the dashboard, rehearse, export, and share with accepted friends.

2. Main Application Areas

AREA	PURPOSE	KEY FILES
Dashboard	Song list, search, sort, column visibility, insights, tag filtering, setlist sharing, TXT/JSON export, Spotify and YouTube playlist export.	`Front/src/Pages/Dashboard`
New Song / Edit Song	Create and update song metadata, instrument links, embedded videos, progress, and setlist tags.	`Front/src/Pages/NewSong`, `Front/src/Pages/EditSong`
Presentation	Live performance view with chord/lyrics rendering, toolbox controls, video tools, scrolling, and chord support.	`Front/src/Pages/Presentation`
Calendar	Events, friend-only invitations, notifications,	`Front/src/Pages/Calendar`

AREA	PURPOSE	KEY FILES
	invitation response flow, and shared rehearsal scheduling.	
Profile / Friends	User profile, avatar, account settings, friendship invitations, accepted friends, and friend revocation.	`Front/src/Pages/UserProfile`, `Front/src/Tools/modal/modalContent/Invitations.jsx`
Tools	Tuner, metronome, chord library, and utility routes.	`Front/src/Pages/Tuner`, `Front/src/Pages/Metronome`, `Front/src/Pages/ChordLibrary`

3. Architecture

The application is split into a React frontend, a Node.js API, Python scraping/services, MongoDB persistence, and supporting Socket.IO communication.

LAYER	TECHNOLOGY	RESPONSIBILITY
Frontend	React, Vite, Tailwind CSS, Axios, Socket.IO client	Authenticated user interface, dashboard, tools, notifications, calendar, song editing, and playlist export flows.
Backend API	Node.js, Express, MongoDB driver, JWT, Socket.IO	Authentication, user data, song library operations, friendship system, notifications, calendar events, setlist shares, and YouTube integration routes.
Scraper Services	Python	Scrape and normalize external song/chord/lyrics data from supported providers.
Database	MongoDB	Stores auth users, song libraries, notifications, invitations, calendar events, setlist shares, user logs, and general song data.
Browser Extension	Chrome Extension	Captures song information from supported sites and sends structured song drafts into the platform.

4. Dashboard Options

The dashboard options panel is the main control surface for filtering, inspecting, exporting, and sharing the visible song library.

Desktop Layout

1. First row: Insights.
2. Second row: Tags / Setlists on the left and Columns Data on the right.
3. Third row: Export on the left and Playlists on the right.

Mobile and Tablet Layout

On iPhone and iPad sized screens, every option section is rendered as a closed accordion by default. Users expand only the section they need, reducing the height of the filter panel.

5. Insights

The Insights box summarizes the current visible song set:

- Total visible songs.
- Average progress ratio.
- Ready songs at 100% progression.
- Not-started songs at 0% progression.
- Main instrument based on the visible songs.
- Instrument counts for Guitar 1, Guitar 2, Bass, Keys, Drums, and Voice.

6. Tags and Setlists

Tags and setlists are currently represented through the song `setlist` array. Dashboard filters use these tags to decide which songs should remain visible.

The Tags / Setlists box has two tabs:

- **Tags:** create, edit, remove, and select setlist tags for filtering.
- **Share:** share the selected setlists with accepted friends.

> Any tag selected in the Tags tab automatically appears in the Share tab. When no tags are selected, the share tab treats all available setlists as exportable candidates.

7. Setlist Sharing

Setlist sharing follows the same trust model as Calendar invitations: users can only share setlists with accepted friends.

Sender Flow

1. User opens Dashboard Options.
2. User selects one or more setlist tags in the Tags tab.
3. User opens the Share tab.
4. The email input suggests accepted friends from the current user profile.
5. User selects or types the destination friend email.
6. User clicks Send.
7. The frontend calls `POST /api/setlist-shares`.

Backend Share Creation

The server validates the request and builds the shared payload from trusted stored data:

- Checks that the sender is authenticated.
- Checks that the recipient exists.
- Blocks sharing to self.
- Checks that sender and recipient are accepted friends.
- Finds sender songs containing the selected setlist tags.
- Stores a pending setlist share record in MongoDB.
- Creates a notification for the recipient.

Recipient Flow

1. Recipient receives a notification.
2. Recipient clicks the notification.
3. A modal shows the sender, setlist names, and song count.

4. Recipient accepts or declines.
5. On accept, the server imports the songs into the recipient library.
6. The modal closes and the page reloads so the new setlist appears immediately.

Import Behavior

- If a received song does not exist in the recipient library, it is inserted.
- If the song already exists by same artist and song title, the setlist tag is merged into the existing song.
- Received setlist names are added to the recipient available setlists.
- Sender receives a notification when the share is accepted or declined.

8. Notifications

Notifications are loaded from `/api/notifications` and also pushed live through Socket.IO using the user email as the room. Current notification categories include:

- Friendship invitations and responses.
- Calendar invites and event updates.
- Setlist share requests and share responses.

Notification click handling routes calendar invitations to the calendar page and opens the setlist share modal for setlist-share notifications.

9. Calendar and Friendship Model

The calendar feature introduced the accepted-friends permission model now reused by setlist sharing. A user can only invite another user to a calendar event or share a setlist when the recipient is already in their accepted friendship list.

The friendship flow includes:

- Search registered users.
- Send friend invitations.
- Accept or decline invitations.
- Persist accepted friendships on both users.
- Notify both sides about relevant friendship changes.

10. Playlist Export

The dashboard Playlists box creates external playlists from the currently visible songs. It supports Spotify and YouTube flows.

- Spotify export uses OAuth and stores payload in session storage during redirect.

- YouTube export uses popup OAuth and posts normalized songs to the backend YouTube endpoint.
- The playlist name defaults to a date/count based name.

11. Visible Song Export

The Export box downloads visible dashboard songs as TXT or JSON. This is separate from setlist sharing:

- **TXT:** lightweight ordered list for quick sharing or printing.
- **JSON:** structured export of the visible song objects.

12. Backend API Highlights

ENDPOINT	PURPOSE
`GET /api/me`	Returns the authenticated safe user profile.
`GET /api/users/search`	Search registered users by email.
`GET /api/notifications`	Lists current user notifications.
`PUT /api/notifications/:id/read`	Marks one notification as read.
`POST /api/invitations`	Sends a friendship invitation.
`PUT /api/invitations/:id/respond`	Accepts or declines a friendship invitation.
`POST /api/calendar/events`	Creates an event and sends friend-only calendar invites.
`PUT /api/calendar/events/:id/respond`	Accepts or declines a calendar invite.
`POST /api/setlist-shares`	Creates a pending setlist share and notifies the recipient.
`GET /api/setlist-shares/:id`	Fetches a share for sender or recipient.
`PUT /api/setlist-shares/:id/respond`	Accepts or declines a setlist share.
`PUT /api/updateSetlists`	Synchronizes available setlists and removes deleted tags from songs.

13. Data Model Summary

COLLECTION	ROLE
`authUsers`	Authentication records and accepted friendship metadata.
`data`	Main user song library documents. Each document stores `email`, `userdata`, and setlist metadata.
`notifications`	Notification records shown in the notification bell and pushed through Socket.IO.
`invitations`	Friendship invitation lifecycle records.
`calendarEvents`	Calendar events and pending/accepted invited users.
`setlistShares`	Pending, accepted, and declined setlist sharing records with shared song payloads.
`userLogs`	Auditable user activity logs for profile/log views.

14. Frontend Routing

The React app defines protected routes for the main workspace:

- `/` Dashboard
- `/newsong` New song
- `/editsong/:artist/:song` Edit song
- `/presentation/:artist/:song/:instrument` Presentation
- `/calendar` Calendar
- `/metronome` Metronome
- `/tuner` Tuner
- `/chordlibrary` Chord library
- `/userprofile/:userid` User profile

15. Development Commands

AREA	COMMAND	DESCRIPTION
Frontend	`npm run dev`	Starts Vite dev server.
Frontend	`npm run build`	Builds production frontend assets.
Frontend	`npm run lint`	Runs ESLint. Currently blocked by invalid `.eslintrc.cjs` key `REMOVED_MONGO_USER`.
Node API	`npm start`	Starts the Express API from `Server/liveNloud/node`.
Server syntax	`node --check Server/liveNloud/node/index.js`	Checks Node server syntax.

16. Current Verification Status

- `npm run build` passes for the frontend.
- `node --check Server/liveNloud/node/index.js` passes.
- `npm run lint` is blocked by an existing invalid ESLint config property: `REMOVED_MONGO_USER`.

17. Recommended Next Improvements

- Fix the ESLint configuration so linting can run again.
- Add automated tests for setlist sharing accept/decline behavior.
- Add a share history view for sent and received setlists.
- Add a clear imported-success screen after accepting a setlist instead of relying only on page reload.
- Consider moving large dashboard components into smaller subcomponents as the options panel grows.
- Add backend indexes for `setlistShares.recipientEmail`, `setlistShares.senderEmail`, and `setlistShares.status`.

18. Recent Change Summary

- Refined dashboard options into a three-row desktop layout.
- Added mobile/tablet closed-by-default accordions for filter sections.
- Improved Insights presentation and instrument labels.
- Added Tags and Share tabs inside Tags / Setlists.
- Connected selected tags to the Share tab.
- Added friend suggestions to the share email input.
- Implemented full backend setlist share lifecycle.
- Added notification modal for accepting or declining received setlists.
- Imported accepted setlist songs into the receiver library and reloaded the page after acceptance.